SMALL LANGUAGE MODELS · DEEP DIVE · COMPANION TO THE FIELD CARD

When Prompting Runs Out

The class says fix the architecture before you buy parameters. This is what to do in the minority of cases where architecture is already right and the model still won't conform, a complete fine-tuning run, with the decisions and the traps, drawn from a real project.

18 → 97% Schema conformance	**~1,100** Training pairs	**~80 min** On a laptop	**$0** GPU spend
2.1 GB Shipped model			

QUESTION 06

Where this sits in the decision sequence

The field card gives you five questions: task shape, `N`, volume, a non-cost reason to self-host, and whether you have a golden set. Fine-tuning is question six, and it is only reachable once the first five have been answered honestly.

The reason is the class thesis restated: **fine-tuning does not raise per-step reliability the way decomposition does.** Cutting `N` from twenty steps to four verified segments moves your end-to-end success rate more than any adapter will. If you fine-tune to avoid decomposing, you have bought a smaller gain at a much higher maintenance cost, you now own a model, a dataset, an eval harness, and a retraining obligation forever.

So reach for it only when all of the following hold:

- The task is decomposed already, and each step is verified.
- A cheap structural fix hasn't worked: constrained decoding, a grammar, few-shot examples, a repair-and-retry pass, or a validator that escalates to a bigger model.
- The remaining failure is **shape, not thinking**, see the test below.
- You have a golden set, because it is about to become both your training data and your evidence.

DECISION · THE CLASSIFICATION THAT GOVERNS EVERYTHING

Format, domain, or reasoning?

Format. The model understands the task but emits the wrong shape, invalid JSON, wrong field names, a chatty preamble that breaks your parser. Ideal fine-tuning target. Converges fast, on few examples, and improves dramatically.

Domain. Well-formed output that doesn't use your vocabulary or conventions. Fine-tuning helps moderately, with a few hundred good examples.

Reasoning. The model genuinely cannot work the answer out. **Fine-tuning will not fix this.** Decompose further, add tools or retrieval, or escalate to a bigger model.

```
Test: can a peer model of the same size do it? If yes → format. If no
model at that size can → reasoning.
```

That test is the cheapest diagnostic available and it costs one afternoon. In the project this guide is drawn from, one 3B model produced malformed structured output while a similar-sized model succeeded on identical prompts. That single comparison reclassified the problem from "this model is too small" to "this model doesn't know our shape", and only the second is fixable by training.

> Decomposition buys reliability. Fine-tuning buys conformance. Do not use one to pay for the other.

STEP 01

Pick the base, then prove the toolchain

Choose the base model before building data, it fixes your memory ceiling, your tooling, and your licence story.

CRITERION	WHY IT DECIDES THINGS
Licence	Permissive licences sometimes carry an extra use-policy layered on top. Read both.
Architecture	Determines which layers an adapter can attach to. A hybrid or state-space model silently trains almost nothing under an attention-only config.
Size	Sets training memory and serving speed. Respect the class's floors: sub-4B for extraction and classification, 7–9B before agentic tool use.
Tooling maturity	A model your stack cannot load is worth zero, whatever the leaderboard says.
Behaviour switches	Reasoning modes change output shape entirely. You must be able to turn thinking off for structured output.

TRAP · YOUR KNOWLEDGE OF THE MODEL LANDSCAPE IS STALE

Model families change architecture between generations. In this project the plan named a model that had been superseded weeks earlier, and the newer generation was a plain dense transformer where the previous one had been a hybrid design. The adapter configuration that was correct for one would have quietly under-trained on the other.

Rule: verify the current release and read the architecture section of the model card, every time. Never carry a model choice forward from a document written months ago.

Then smoke-test the whole chain on throwaway data

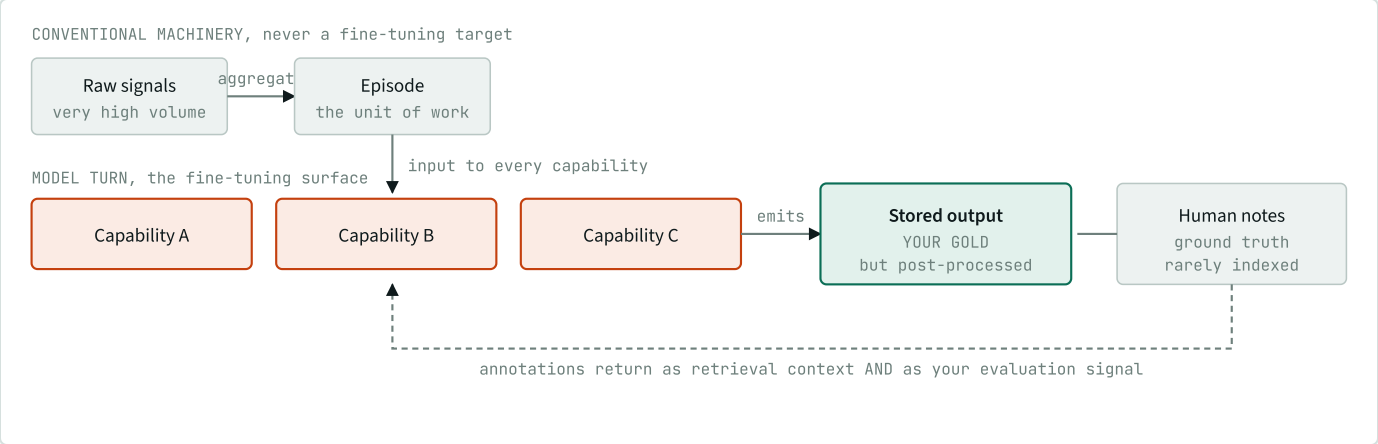
The most common way these projects die is discovering three weeks in that some link doesn't support your model. Before building any dataset, push a few dozen junk examples all the way through: convert the base, train fifty steps, merge the adapter, convert to your serving format, quantise, start the server, get one completion back. You are not testing learning. You are testing that every link exists. An hour here routinely saves a week.

> TRAP · SILENT EMPTY OUTPUT
>
> A reasoning-capable model asked a trivial question returned *empty* text. Nothing was broken: it had spent its whole token budget thinking, and the server put that in a separate field. When a model returns nothing, check for a reasoning field before debugging anything else, and confirm your "disable thinking" switch is actually being honoured, because a server that isn't configured for it will accept the option and ignore it.

STEP 02

Find the gold, and learn what was actually stored

Training data for an existing capability usually already exists: it is the output your current system has been producing all along. Most operational products share one shape, conventional machinery aggregates raw signals into an episode, a model turn consumes that episode, and humans annotate the result afterwards.



The shape most operational AI products share. Aggregation and correlation are conventional machinery, not fine-tuning targets. The green box is your gold, and the caveat on it is the trap below.

> TRAP · STORED OUTPUT IS NOT MODEL OUTPUT
>
> The records in a datastore are usually not what the model emitted. They are what the system stored *after* parsing, validating, renaming fields and adding envelope metadata. In this project, field names had been converted between naming conventions and routing identifiers added that no model ever produced.
>
> Train on stored records and you teach the model the *database's* shape rather than the shape your parser accepts, a poisoning that passes every check you write against the database and fails in production.
>
> **Rule:** find the exact function where the model's raw text is parsed. Whatever goes *into* it is your training target. Write an explicit transform from stored shape back to emitted shape and verify one example by hand.

Interrogate any candidate gold source

- **Raw emission or processed artefact?** Assume processed until proven otherwise.
- **Is it actually good?** Output from a strong model that drove real successful outcomes is trustworthy. Unchecked output is just old text.
- **Is the matching input recoverable?** Outputs without inputs are half a dataset, and this is usually the hard part.
- **How much is there?** Hundreds for conformance, thousands for domain adaptation. Count before you build.

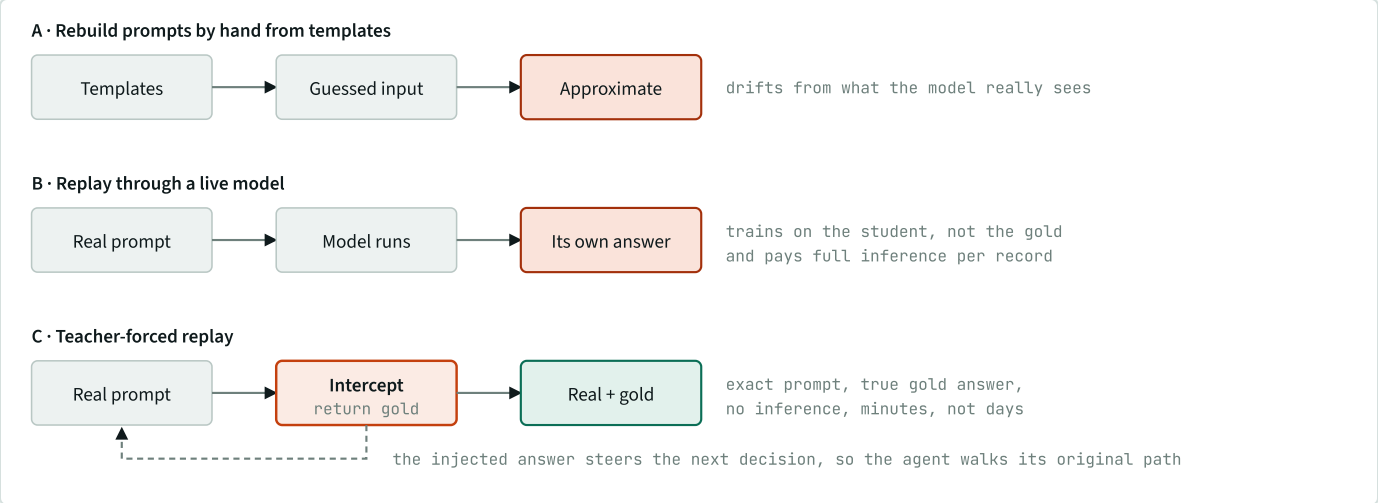
- **Privacy and licence?** Operational data needs scrubbing and a defensible handling story before it becomes weights.

STEP 03

Recovering the prompts

Supervised fine-tuning needs *(prompt, answer)* pairs. Systems log what they produced, almost never what they asked, in this project there were thousands of stored answers and **zero** stored prompts, because the prompt logger kept data in memory only and cleared it every few minutes.

So the prompts must be regenerated. There are three ways, and the choice sets your dataset's fidelity.



The options differ in *where each half of the pair comes from*. Only C gets the real prompt and the true gold together. The dashed edge is the mechanism: feeding back the recorded answer makes a branching agent take the branches it originally took.

DECISION · PAIRING METHOD

Teacher-forced replay, whenever a runnable system exists

Run the real system over reconstructed inputs, but intercept the one function that calls the model. Instead of calling out, return the recorded answer, and log

the prompt the system just built alongside it. Byte-exact prompts, genuine gold, no inference cost, minutes instead of days.

Key the lookup by **step type and subject**, never by position, so divergence cannot silently misalign pairs. When no gold matches a call, fall through or skip, **never fabricate a pair**. Reconcile expected against produced counts per step type afterwards; a step yielding zero while gold exists is a reconstruction bug, not missing data.

```
Use A only with no runnable system · B only with no stored gold
```

Be honest about reconstruction fidelity. Where a rebuilt input carries forward the original system's own conclusions as "evidence", the prompt contains part of its own answer. Those pairs teach **format faithfully but leak for reasoning**. When the goal is explicitly a format fix, that is an acceptable trade, but write it down, so nobody later reads your accuracy numbers as reasoning gains.

STEP 04

Gate, balance, split, scrub

Raw pairs are not a dataset. Five transforms stand in between, and skipping any one produces a model that scores well and behaves badly.

Validate against the real schema. Not "does it parse", does it satisfy the contract, with required fields and enumerated values in range? Import the real validators from the consuming application. This run rejected 12.6% of pairs; every one would have taught the model something the parser refuses. Strip code fences before validating, or you will reject everything and conclude your gold is garbage.

Deduplicate. Replays and retries produce near-identical pairs that inflate size, bias training, and leak across the split.

Balance. Cap a dominant class at a defined share, but don't over-correct: with only two classes, a 30% cap discards most of your data. This run capped at 60% and landed at 54/46.

Split by episode. The one that is most often botched, and the one that silently inflates every number you report, see the trap below.

Scrub. Hostnames, addresses, tokens and identifiers become effectively unremovable once trained in. Scrub on the way into the dataset, not later.

> TRAP · ROW-LEVEL SPLITTING IS LEAKAGE
>
> Rows drawn from the same episode share context. Split rows randomly and near-identical content lands on both sides of the wall, your validation set measures memorisation and reports it as generalisation.
>
> **Rule:** split at the level of the underlying episode, incident, conversation, document, user, and hold out whole episodes. This is the most commonly botched step in the process, and it silently inflates every number you report.

STAGE	COUNT	NOTE
Raw pairs harvested	1,283	two step types were enough
Failed schema gate	-162	12.6%, invalid enum or missing field
Duplicates	-4	replay artefacts
Kept	1,117	across 513 episodes
Train / validate	1,001 / 116	51 episodes held out whole

Set the volume gate *before* harvesting so you know when to stop. A few hundred clean examples move conformance; a thousand is comfortable. This run targeted 800 and finished at 1,117 using two of the six recorded step types, the rest were dropped deliberately once the target was met.

STEP 05

The training run

The part everyone imagines is the project. It is the easiest phase and, by now, nearly mechanical.

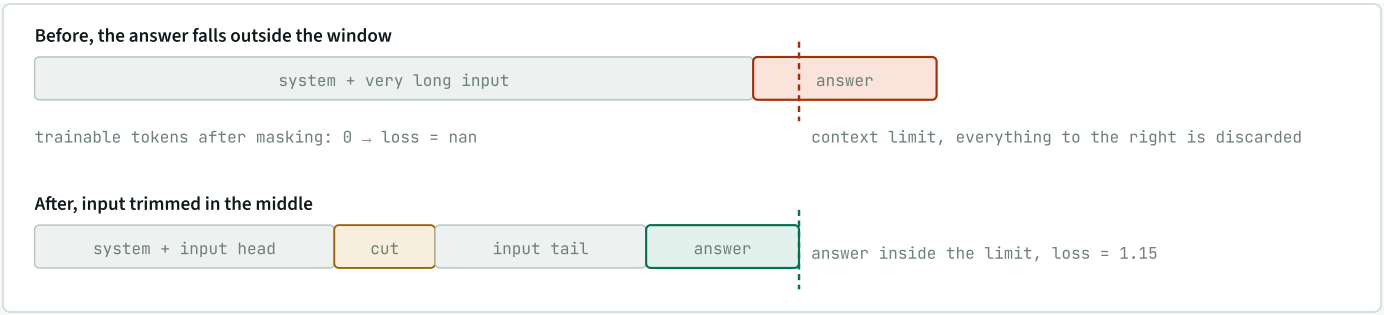
SETTING	VALUE	REASONING
Method	QLoRA (4-bit base)	Small adapter over a quantised base, fits consumer memory
Rank	`32`	Ample for format learning; 8–16 also works
Layers adapted	`16`	Full depth exhausted memory for no gain on a dense model
Loss	Answer only	Mask the prompt, or capacity goes to memorising inputs
Steps	`2,000`	~2 epochs over 1,001 examples
Batch / context	`1 / 4,096`	Set by the memory ceiling, not preference
Peak memory	`24.4 GB`	On a 36 GB laptop

Loss fell 1.15 → 0.31, best slightly before the end with a small uptick after, ordinary over-fitting onset, and the reason to checkpoint periodically instead of trusting the last step. **For conformance work, loss is only a proxy.** The number that matters is the share of held-out outputs that actually validate; generate and check on a schedule.

TRAP · THE ONE THAT NEARLY STOPPED EVERYTHING

Training reported a loss of `nan` immediately. The chain: prompts were evidence-heavy, some over 5,700 tokens. The trainer truncates *from the end*. The end is where the answer lives, so the answer was cut off. Prompt-masking then masked everything remaining. A sequence with zero unmasked tokens divides by zero, loss becomes `nan`, nothing trains.

Fix: trim the *middle of the input* during dataset construction so the system prompt and the answer always survive the window. **Rule:** if you mask prompts, guarantee every example retains answer tokens after truncation, and check that the first validation loss is a real number before committing to a full run.



Same context limit, same masking setting. The only change is where truncation lands.

> TRAP · MEMORY IS A WALL, NOT A SLOPE
>
> Adapting every layer at a larger batch exhausted memory outright, a hard crash, not a slowdown. Settings copied from a tutorial written for different hardware fail immediately. Start conservative, confirm the run is healthy, then scale up if you have headroom.

STEP 06

Evaluating without fooling yourself

An absolute score means little alone. **Always measure the base model on the identical held-out set with the identical harness.** Without that column you cannot separate what you achieved from what the model could already do.

MODEL	STEP TYPE A	STEP TYPE B	OVERALL
Model 1 (smaller), base	2%	30%	18%
Model 1 (smaller), fine-tuned	98%	95%	97%
Model 2 (larger), fine-tuned	98%	95%	97%
Model 2 (larger), base	0%	0%	0%*

One row carries a caveat that matters more than the headline. * That 0% is not proof of incapability: the model defaults to a reasoning mode and returned no

parseable content within the budget. The honest reading is "unusable out of the box for this task", and reporting a clean zero without the note would mislead.

Both fine-tunes landing on *identical* figures is itself the most reusable finding: the dataset, not the base model, was doing the work.

DECISION · PICKING A WINNER WHEN QUALITY TIES

Decide on the secondary axes, and benchmark before committing

Equal accuracy meant the choice fell to size (2.1 GB vs 4.9 GB) and CPU generation speed (52 vs 36 tokens/sec). The smaller model won on every remaining axis. Measure throughput against the class's floor, below roughly 5 tok/s an interactive agent stops being tolerable, *before* you commit.

Test what you did not train

A specialised model can lose general ability. Summarisation, deliberately not trained, was measured before and after: fabricated-detail rate rose from 36% to 48%. The fine-tune had mildly degraded a capability it never touched.

That finding is worth as much as the headline: **one dataset buys exactly one capability.** A second capability needs its own data and its own gate, and you may quietly lose ground you already had.

Two honesty rules for any number you publish. Name your proxy, this fabrication metric flagged any figure absent from the source, which over-counts reformatting, so only the *relative* movement is trustworthy. And hold every model to identical conditions; this run nearly recorded a 2-of-25 failure that was purely a thinking-mode artefact of the harness itself.

STEP 07

Shipping it

- **Merge, convert, quantise.** Fold the adapter into the base and quantise, 4-bit on a 3B model lands near 2 GB with negligible loss for conformance tasks.
- **Verify the chat template survived conversion.** A mismatch between training and serving formats degrades quality silently, no error, just worse output.
- **Record the serving flags.** Undocumented runtime flags are a multi-hour mystery for whoever deploys next.
- **Name the model inside the file,** not just the filename, the internal name is what serving APIs report and what your router matches on.
- **Deploy alongside, not instead.** Route by model identifier so comparison and rollback are config changes, not redeployments.
- **Keep the eval in CI.** The class's Lab 5 harness is exactly the thing that tells you when a model or prompt change regressed you.

REFERENCE

Every trap in one table

SYMPTOM	ACTUAL CAUSE	FIX
Empty model output	Reasoning mode consumed the budget	Disable thinking; confirm the server honours it
Loss is `nan` at step one	Truncation removed the answer; masking left nothing	Trim the input's middle so the answer survives
Out of memory, hard crash	Too many adapted layers, batch too large	Fewer layers, batch of one, then scale
Adapter trains but barely learns	Attention-only targets on a hybrid architecture	Check architecture; target all linear layers
Every gold answer fails validation	Answers wrapped in code fences	Strip fences before validating

SYMPTOM	ACTUAL CAUSE	FIX
Suspiciously excellent scores	Row-level split leaked shared context	Split by episode, hold out whole episodes
Model emits fields the parser rejects	Trained on stored records, not raw emissions	Transform stored shape back to emitted shape
A step type yields no pairs	Its inputs were never stored, only outputs	Reconcile expected vs produced, per step type
Long job dies repeatedly	No resumability; restarts redo everything	Make it resumable and append-only from the start
Confusing cache errors	Interrupted download left a partial snapshot	Re-fetch to complete before converting

One operational lesson that isn't about models at all: the harvest in this project was killed by its environment roughly every hundred records, and finished anyway across repeated passes because it was **resumable and append-only**. Retrofitting that cost minutes; not having it would have cost the project. Any job over a few minutes should be resumable before you first run it.

REFERENCE

The checklist

- [ ] Answer the field card's five questions first. Fine-tuning is question six.
- [ ] Confirm the architecture is decomposed and each step verified before training anything.
- [ ] Exhaust constrained decoding, grammars, few-shot, repair passes and routing.
- [ ] Classify the failure: format, domain, or reasoning. Only the first two are trainable.
- [ ] Run the peer-model test, proof the capability exists at that size.
- [ ] Verify the current model release and read its architecture section.

- [ ] Smoke-test the entire chain on throwaway data before building a dataset.
- [ ] Locate gold; determine whether it is raw emission or processed artefact.
- [ ] Find the parser, whatever enters it is your training target.
- [ ] Prefer teacher-forced replay when a runnable system exists.
- [ ] Validate against real schemas; strip code fences first.
- [ ] Deduplicate, balance classes, split by episode, never by row.
- [ ] Scrub sensitive data before it becomes weights.
- [ ] Guarantee the answer survives truncation if you mask prompts.
- [ ] Confirm the first validation loss is a real number.
- [ ] Measure the base model on the identical held-out set.
- [ ] Run a regression check on capabilities you did not train.
- [ ] Benchmark throughput against your latency floor before committing.
- [ ] Document serving flags, model naming, and every trade-off accepted.

THE SHORT VERSION

> **Fine-tuning was the easy night's work. Finding trustworthy gold, recovering the prompts that went with it, and refusing to fool ourselves at evaluation, that was the project.**

If you take two habits from this page, take the failure classification at the top and the base-model column in the evaluation table. The first stops you training something that needs a bigger model or a better decomposition. The second stops you claiming credit the base model already deserved.

Companion to the SLM Field Card and the two-hour class. Figures and numbers come from one real production run, anonymised.